AI Generalist

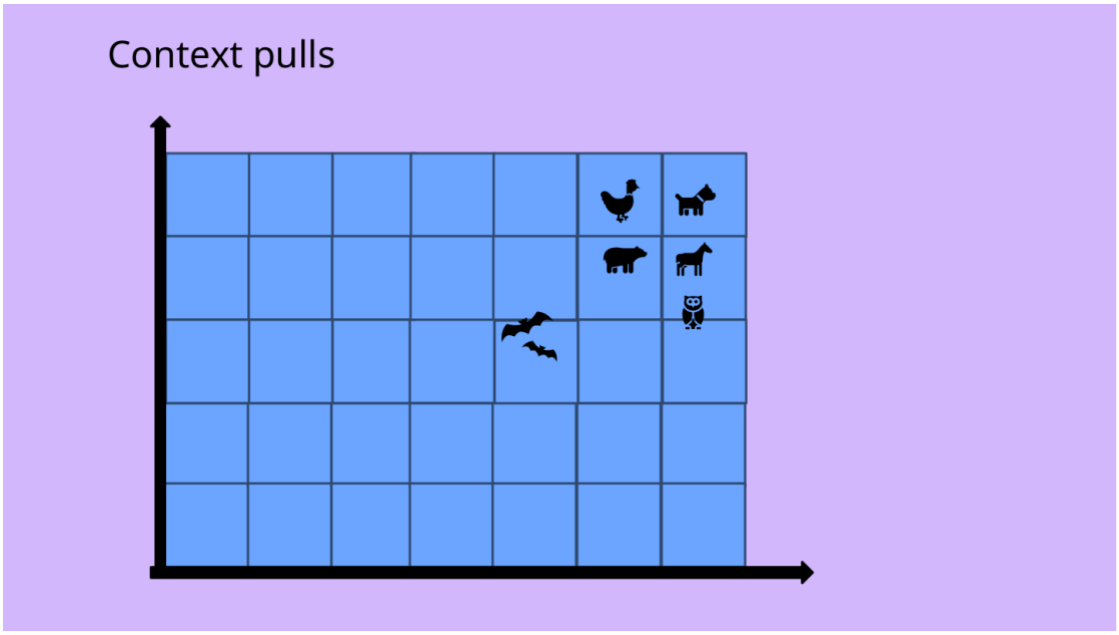
Python Vibe-Coding Lab: Cosine Similarity, Attention, and Mini-RAG

- Use an AI coding assistant to generate, run, test, and explain three small Python applications. The major skills are: prompt, run, test, debug, and reflect.

Lab Snapshot

Item	Recommendation	Why it matters
Time	75-90 minutes	Enough time for prompting, running code, debugging, and reflection.
AI tool	Primary: ChatGPT. Optional new-tool path: Cursor Hobby plan if students already can install it.	ChatGPT is familiar; Cursor exposes students to AI inside a code editor. Cursor currently lists a free Hobby plan with limited Agent requests and limited Tab completions, with no credit card required.
Coding environment	Anaconda Navigator/Spyder/Jupyter, VS Code, or Terminal	The backup code uses only the Python standard library: no pip installs, no API keys, no paid model calls.
Main deliverable	Three working Python mini-apps plus one reflection table	Students see GenAI concepts as runnable systems rather than slide vocabulary.
Safety boundary	No student records, no private files, no API keys	The goal is concept exposure, not connecting live services or using paid tokens.

What this lab supports from the lecture
The uploaded lecture moves from Generative AI to attention, cosine similarity, context pulling the meaning of ambiguous words, scaled dot product attention, and RAG. This lab turns those lecture ideas into three simple Python applications.



Lecture anchor: context pulls meaning. The same word can be pulled toward different meanings by surrounding words.

Part 0 - Set Up the Vibe-Coding Workflow

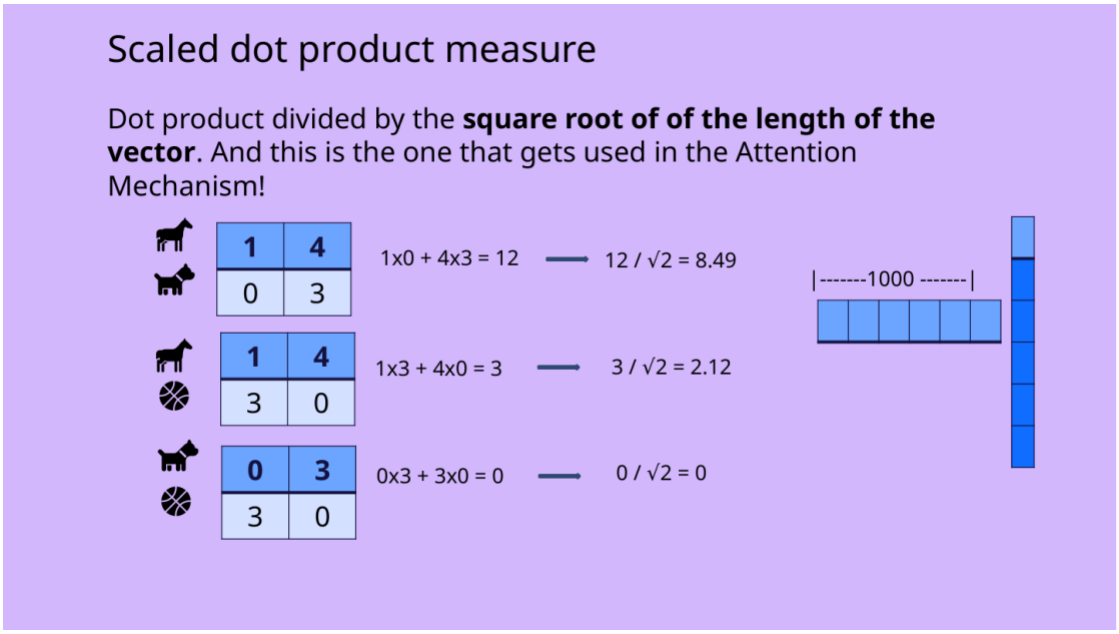
1. Open ChatGPT in one browser tab. This is the default AI coding assistant for the lab.
2. Optional: open Cursor if your instructor has approved it and your machine can install it. Cursor is useful because the AI assistant sits directly inside the code editor.
3. Open your coding environment: VS Code, Anaconda Navigator, Spyder, Jupyter, or a basic Terminal.
4. Create a folder named week4_genai_python_lab.
5. For each part of the lab, ask the AI tool to generate one Python file. Save each file in your lab folder.
6. Run the file. If it works, test it with your own inputs. If it fails, paste the error message back into the AI tool and ask for a fix.
7. If the AI-generated version still fails after two repair attempts, ask your professor for the backup file for that part.

Aha moment
Vibe coding does not mean blindly trusting generated code. It means describing the product clearly, testing it immediately, and using the AI as a coding partner rather than a magic vending machine.

How to Run the Python Files

Environment	Numbered steps	Beginner tip
VS Code	1. Open the lab folder. 2. Open the .py file. 3. Click the Run triangle or open Terminal. 4. Type: python filename.py	Use the VS Code terminal at the bottom so input prompts work.
Anaconda Navigator / Spyder	1. Open Spyder from Anaconda. 2. Open the .py file. 3. Click Run. 4. Use the console for typed input.	Spyder is friendly for beginners because the code and console are visible together.
Anaconda Prompt / Terminal	1. Navigate to the folder. 2. Type: python part1_cosine_similarity.py 3. Press Enter. 4. Follow the prompts.	If python does not work, try python3 on Mac/Linux.
Jupyter Notebook	1. Create a new notebook. 2. Copy the code into a cell. 3. Run the cell. 4. For input-heavy code, a .py file is usually easier.	Jupyter is great for experiments, but command-line apps are cleaner for this lab.

Part 1 - Vibe-Code a Cosine Similarity Calculator



Lecture anchor: scaled dot product measure. Students start with dot product and cosine similarity before moving to attention.

Plain-English idea
Embeddings are coordinates in a meaning-space. Cosine similarity asks: are these two arrows pointing in roughly the same direction? A score near 1 means similar direction. A score near 0 means weakly related. A negative score means the directions are opposed in the toy space.

1. Open ChatGPT or your approved AI coding assistant.
2. Copy the prompt below exactly and paste it into the AI tool.
3. Ask the AI tool to create a Python file named **part1_cosine_similarity.py**.
4. Copy the generated code into VS Code, Spyder, or your preferred Python environment.
5. Run the file.
6. Test the built-in examples first.
7. Then enter your own vectors such as 1,4 and 4,1.

8. Record one result where cosine similarity is high and one result where it is close to zero.

Copy/Paste Prompt 1 - Cosine Similarity App

```
Act as a patient Python teaching assistant. Create a beginner-friendly Python 3 command-line
application named part1_cosine_similarity.py.
Goal:
Teach cosine similarity using simple vectors, inspired by a GenAI lecture on embeddings and
similarity.
Requirements:
1. Use only the Python standard library. Do not use numpy, pandas, scikit-learn, or any external
package.
2. Include functions for dot_product, magnitude, and cosine_similarity.
3. Include built-in examples using a two-dimensional meaning-space: [mammal, sport].
4. Use example vectors such as bat_animal=[4,1], bat_sport=[1,4], cave=[5,0], baseball=[0,5],
dog=[5,0], and ball=[0,5].
5. Print the cosine similarity for several examples and explain the result in plain English.
6. After the examples, allow the user to enter two comma-separated vectors, such as 1,4 and 4,1.
7. Validate the input and show a helpful error message if the vectors are different lengths or not
numeric.
8. Add comments throughout the code so a beginner can understand it.
9. Include if __name__ == "__main__": main().
Important teaching tone:
Explain that this is a tiny teaching model, not a real production embedding model.
```

Test	Vector A	Vector B	Expected idea
Animal bat vs cave	4,1	5,0	High similarity: animal context
Sports bat vs baseball	1,4	0,5	High similarity: sports context
Dog vs ball	5,0	0,5	Near zero: mostly different axes

Part 2 - Vibe-Code a Tiny Attention Mechanism Demo

Plain-English idea

Attention is like asking, “Which words in this sentence should matter most right now?” If the sentence says “The bat flew out of the cave,” words like flew and cave pull bat toward the animal meaning. If the sentence says “The player swung the bat at the ball,” player, swung, and ball pull it toward sports.

1. Create a new AI chat or continue the same coding chat if it is still focused.
2. Copy and paste Prompt 2 below.
3. Save the generated code as **part2_attention_mechanism.py**.
4. Run the program.
5. Compare the output for: The player swung the bat at the ball.
6. Compare the output for: The bat flew out of the cave.
7. Enter your own sentence with the word bat or bats.
8. Write one sentence explaining how context changed the attention weights.

Copy/Paste Prompt 2 - Tiny Attention Demo

```
Act as a patient Python teaching assistant. Create a beginner-friendly Python 3 command-line
application named part2_attention_mechanism.py.
Goal:
Demonstrate the intuition behind attention using the ambiguous word "bat".
Requirements:
1. Use only the Python standard library. Do not use numpy, pandas, scikit-learn, or any external
package.
2. Create a tiny dictionary of toy word vectors. Each vector should have six dimensions: animal,
sport, motion, place, object, grammar.
3. Include words such as bat, bats, ball, baseball, player, swung, hit, flew, flying, cave, pet,
animal, look, all, those, the, at, with, a, is, out, and of.
4. Tokenize a sentence into lowercase words.
5. Ask the user for a sentence containing bat or bats.
6. For the target word bat/bats, compute cosine similarity between the target vector and every other
word vector.
7. Convert those scores into attention weights using a softmax function.
8. Print a clear table with columns: word, similarity, attention weight.
9. Print a plain-English interpretation listing the meaningful words that pulled most strongly on bat.
10. Include built-in demo sentences:
   - The player swung the bat at the ball
   - The bat flew out of the cave
   - John's pet is a bat
11. Add comments throughout the code explaining that this is a toy attention demo, not a full
transformer.
12. Include if __name__ == "__main__": main().
Teaching tone:
Use the classroom analogy that attention is like a spotlight deciding which nearby words deserve more
attention.
```

Part 3 - Vibe-Code a Mini-RAG Application

Features	RAG	AI Agents
Primary Focus	Knowledge Augmentation	Action and Interaction
Mechanism	Information Retrieval and Integration	Tool Utilization and Decision Making
Strengths	Improved Accuracy, Reliability, and Domain Expertise	Task Completion, Problem Solving and World Interaction
Limitations	Retrieval Performance, Static Context, Limited Interactivity	Tool dependency, Complexity of Agent Design, Ethical Considerations

42

Lecture anchor: RAG focuses on knowledge augmentation, while agents focus on action and interaction.

Plain-English idea

RAG stands for Retrieval-Augmented Generation. Before the system answers, it retrieves relevant chunks from a knowledge base. That makes the answer easier to check, because the answer points back to source material instead of sounding confident from nowhere.

1. Open a new AI coding chat or continue your focused coding chat.
2. Copy and paste Prompt 3 below.
3. Save the generated code as **part3_mini_rag.py**.
4. Run the file.
5. Ask the program: What does attention do in a transformer?
6. Ask the program: What is cosine similarity used for?
7. Ask the program: What is LoRA?
8. Ask one question that is not answered by the tiny knowledge base and observe what happens.
9. Record how the retrieved source chunks changed your trust in the answer.

Copy/Paste Prompt 3 - Mini-RAG App

```
Act as a patient Python teaching assistant. Create a beginner-friendly Python 3 command-line
application named part3_mini_rag.py.
Goal:
Build a mini Retrieval-Augmented Generation demonstration that does not require an API key.
Requirements:
1. Use only the Python standard library. Do not use numpy, pandas, scikit-learn, LangChain,
LlamaIndex, or any external package.
2. Create a tiny knowledge base inside the code using a list of dictionaries. Include five source
chunks:
   - Generative AI
   - Attention
   - Cosine Similarity
   - RAG
   - LoRA
3. Each source chunk should have an id, title, and text.
4. Write a tokenizer that lowercases text, removes punctuation, and removes simple stopwords.
5. Convert each source chunk and the user question into bag-of-words vectors using Counter.
6. Compute cosine similarity between the question and each source chunk.
7. Retrieve the top two chunks and print their ids, titles, similarity scores, and text.
8. Print two outputs:
   - An "ungrounded answer style" warning explaining why a fluent answer without sources can be risky.
   - A "RAG answer" grounded in the best retrieved source chunk.
9. If no source chunk has a useful score, say: "Not found in the provided knowledge base."
10. Let the user keep asking questions until they press Enter on a blank question.
11. Add comments throughout the code so a beginner can understand it.
12. Include if __name__ == "__main__": main().
Teaching tone:
Explain that this is a tiny visible version of the RAG idea, not a production enterprise RAG system.
```

Debugging With the AI Tool

Two-repair rule

Make two or three attempts to repair AI-generated code. After two or three failed repair attempts, use the professor backup code. This avoids spending the whole class stuck in error messages.

1. Copy the entire error message from Python, including the last line.
2. Paste it into the same AI chat that generated the code.
3. Ask: “Fix this code with the smallest possible change. Explain the bug in beginner language.”
4. Run the fixed code again.

5. If the second repair fails, stop and ask for the professor backup file.

Copy/Paste Debugging Prompt
The Python code you gave me produced this error: [paste the full error message here] Please fix the code with the smallest possible change. Do not add new features. Explain the bug in beginner-friendly language. Return the complete corrected Python file.

Student Deliverable

Part	What to submit	Evidence	One reflection sentence
1. Cosine	Screenshot or copied output from two vector tests	One high score and one near-zero score	Cosine similarity helped me see...
2. Attention	Output table for two bat sentences	Top attention words for each sentence	Context changed the meaning of bat because...
3. Mini-RAG	Output from two questions	Retrieved source chunks and RAG answer	The grounded answer was easier to trust because...
Debugging	One note about whether AI-generated code worked immediately	If fixed, include the error and repair prompt	The most useful debugging move was...

Mini-Rubric

Criterion	Points	What earns full credit
AI-assisted code generation	2	Student used the copy/paste prompts and attempted to generate working Python code.
Part 1 output	2	Cosine app runs and student interprets at least two results.
Part 2 output	2	Attention demo runs and student explains how context pulls meaning.
Part 3 output	2	Mini-RAG app runs and student compares ungrounded vs grounded answers.
Reflection and safety	2	Student explains what AI helped with, what needed verification, and why no private data/API keys were used.